FitML: A Fairness-Audited Machine Learning System for Sizing and Intelligent Virtual Try-On in E-Commerce

Pillar 5: Capstone Project

Postgraduate Diploma in Artificial Intelligence and Machine Learning

Debie Galleros · July 2026

Live demo: https://fit-ml.netlify.app
Repository: https://github.com/debiegalleros/fitml-capstone

Contents

1 Problem Framing

Business problem. Online clothing returns run at roughly 35%, versus about 8% in physical stores, costing retailers over $100B annually. Sizing uncertainty is the leading driver, but shoppers also hesitate because they cannot see how a garment will actually look on their own body. FitML addresses both halves of that hesitation: a fairness-audited sizing model answers “will it fit?”, and a generative virtual try-on composites the actual catalog garment onto the shopper's own photo before buying — answering “how will this look on me?”. Color-suitability matching (does this shade complement the shopper) was identified as a natural extension and deliberately scoped out of this submission.

Data science problem. Supervised multi-class classification: given a customer's body measurements and the size they ordered, predict the reported fit outcome — fit, runs small, or runs large — paired with a fairness-audit layer that checks whether predictions are equally reliable across body types, heights, and size ranges. The labels come from real customer fit feedback, not collaborative filtering.

Success metrics. Technical: accuracy and macro-F1 (macro-F1 catches the minority-class failures that raw accuracy hides under a 73%-majority class), plus disparate impact ratio and equalized odds for the fairness layer. Business: projected return-rate reduction (35% toward a 15–20% target), with fewer wrong “it fits” reassurances as the mechanism.

The differentiator. Commercial sizing tools are black boxes. FitML publishes its full bias audit — per-group metrics, disparate impact, equalized odds, SHAP explainability, and an honest before/after mitigation comparison — as a first-class deliverable alongside the product.

2 Data Collection & Understanding

FitML deliberately combines sources with different, clearly-flagged statuses — real audited data where it exists, and a small synthetic extension where it does not:

Need	Source	Status
Model training + full fairness audit (graded core)	Kaggle “Clothing Fit Dataset for Size Recommendation” — 251,047 real customer reviews (58,503 ModCloth + 192,544 RentTheRunway) with self-reported measurements and fit feedback	Real
Men's catalog extension (~11 items, demo only)	200 synthetic customers labeled by rule-based lookup against Uniqlo's published men's size charts	Synthetic population, real sizing standard — excluded from the fairness audit
Garment catalog imagery (visual only, never training data)	Kaggle Fashion Product Images Dataset (MIT license) — 116 curated items, background-removed via rembg, 232 hue-shifted color variants	Real photos, demo pricing

Why this combination. Both ModCloth and RentTheRunway sell women's apparel, so the real dataset is women's-only. Rather than force one dataset to cover both genders — or silently train on invented men's fit feedback — the project uses real, citable data where it is available and a small, explicitly-documented synthetic extension for the gap. The men's extension is a deterministic size-chart lookup, not a trained model, and its ~200-row synthetic population is far too small for valid group comparisons — so it is excluded from the fairness audit by design, an explicit documented limitation rather than a silently patched one.

Key fields (full data dictionary in docs/data_dictionary.md). Height, weight, bust band + cup, hip, self-reported body type (hourglass / pear / apple / athletic / petite / straight & narrow / full bust), garment category, ordered size, and the target fit_feedback (small / fit / large). Waist was dropped: RentTheRunway never collected it and ModCloth's coverage was 3.5% (~96.5% missing), leaving <1% combined coverage — too sparse to impute defensibly. Post-purchase fields (rating, review text) were excluded as features because they would not exist at prediction time.

Catalog imagery provenance. iMaterialist Fashion was evaluated first and rejected: its images are in-the-wild street-style, red-carpet, and runway photos — busy backgrounds, identifiable people, some with third-party photographer watermarks — a poor fit both technically (for background-removal + affine-warp compositing) and ethically. The Fashion Product Images Dataset provides clean, plain-background product shots with structured gender/category/color metadata; only the ~115 needed full-resolution images were fetched via targeted single-file downloads, not the full 24.7 GB archive. Catalog prices are programmatically generated demo pricing within per-category PHP bands, documented as such.

3 Preprocessing, EDA & Feature Engineering

Full walkthrough with reasoning in notebooks/02_eda_feature_engineering.ipynb; the reusable logic lives in src/prepare_data.py so later phases import it instead of duplicating it. Output: data/processed/model_ready.csv — 251,047 rows × 15 columns, no remaining nulls.

Harmonizing two schemas

The sources disagree on units and formats (height as “5ft 6in” vs “5' 8″”, hips in inches, bust as band+cup strings). Parsing normalizes everything to metric. ModCloth rows tagged only new/sale/wedding (24,287 rows, ~29% of ModCloth) carry no garment-category signal and were dropped; RentTheRunway's 68 fine-grained category tags were mapped onto ModCloth's 4-bucket taxonomy (tops / dresses / bottoms / outerwear) as category_broad. Cup letters were unified to an ordinal scale (AA=0 … K=11) using the standard DD=E, DDD=F, DDDD=G equivalence.

Structural missingness, handled honestly

Missingness here is structural, not random: weight is absent for all ModCloth rows (never collected), hips for all RentTheRunway rows, body type for all ModCloth rows. Instead of silently imputing and pretending the values are measured, each such field gets a _missing indicator flag alongside median imputation, so models — and the fairness audit — can distinguish a measured value from an invented placeholder.

body_type gets an explicit not_reported category rather than a guessed shape: inventing a protected-attribute-like label would contaminate the Phase 6 audit. This decision pays off later — SHAP shows the model actually uses these missingness flags (Section 5.3).

Class imbalance in the target

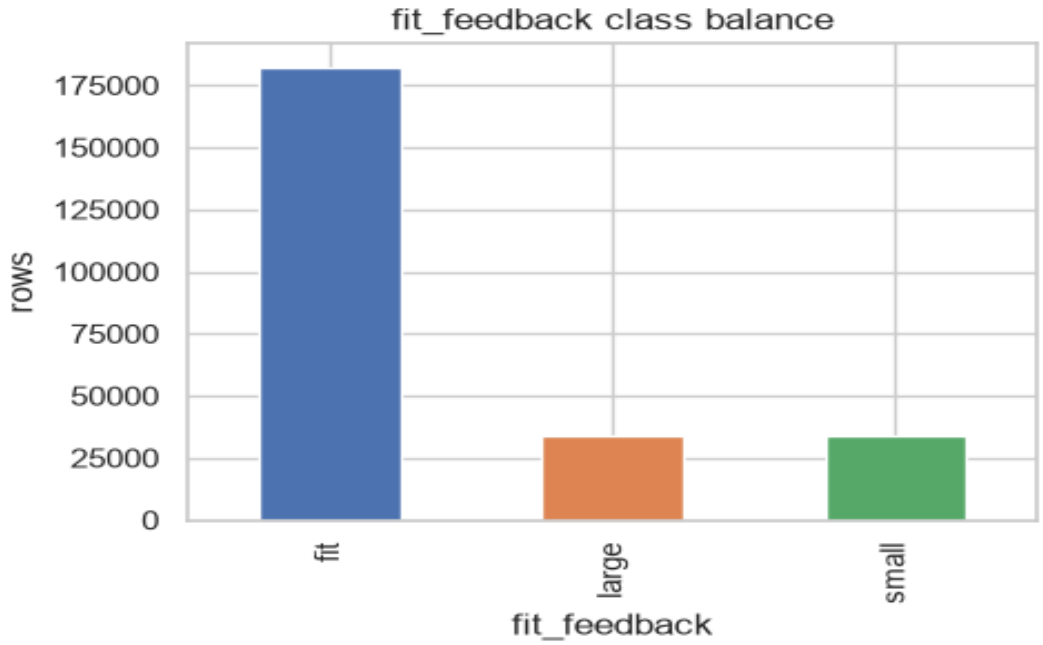

Figure 1. Target distribution: 72.7% fit, 13.7% large, 13.6% small. Severe class imbalance (73%/13%/13%) necessitates macro-F1 as the selection metric; raw accuracy is dominated by the majority class.

The target is ~73/14/13 — exactly why macro-F1, not accuracy, drives model selection: a model predicting “fit” always achieves ~72.7% accuracy while failing entirely on the minority classes that need size corrections.

Distributions and correlations

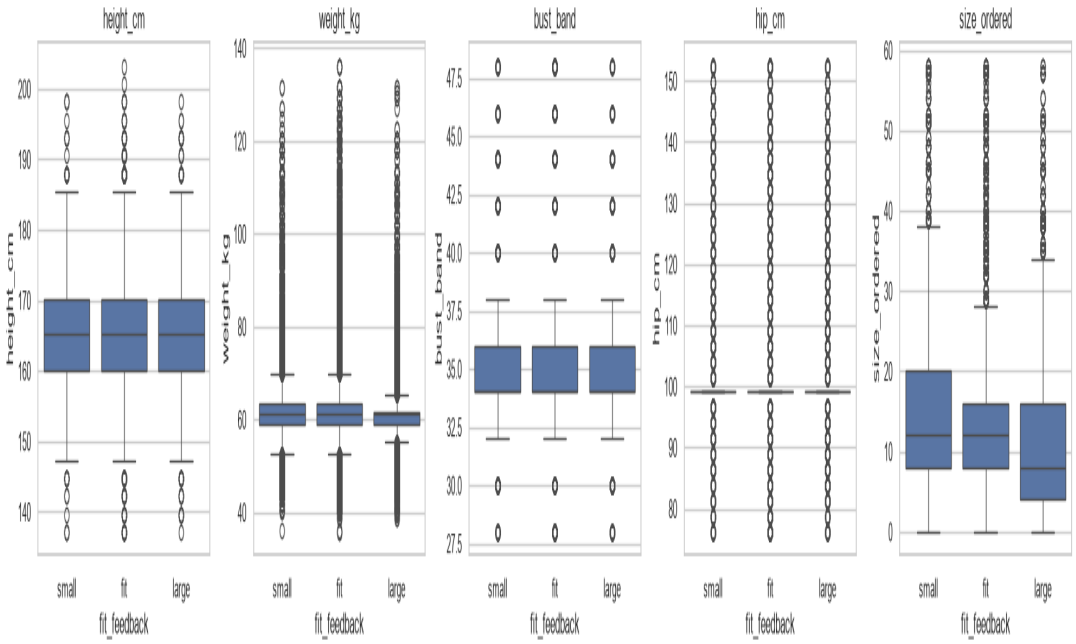

Figure 2. Feature distributions per fit_feedback class. size_ordered separates most clearly; body measurements separate weakly on their own.

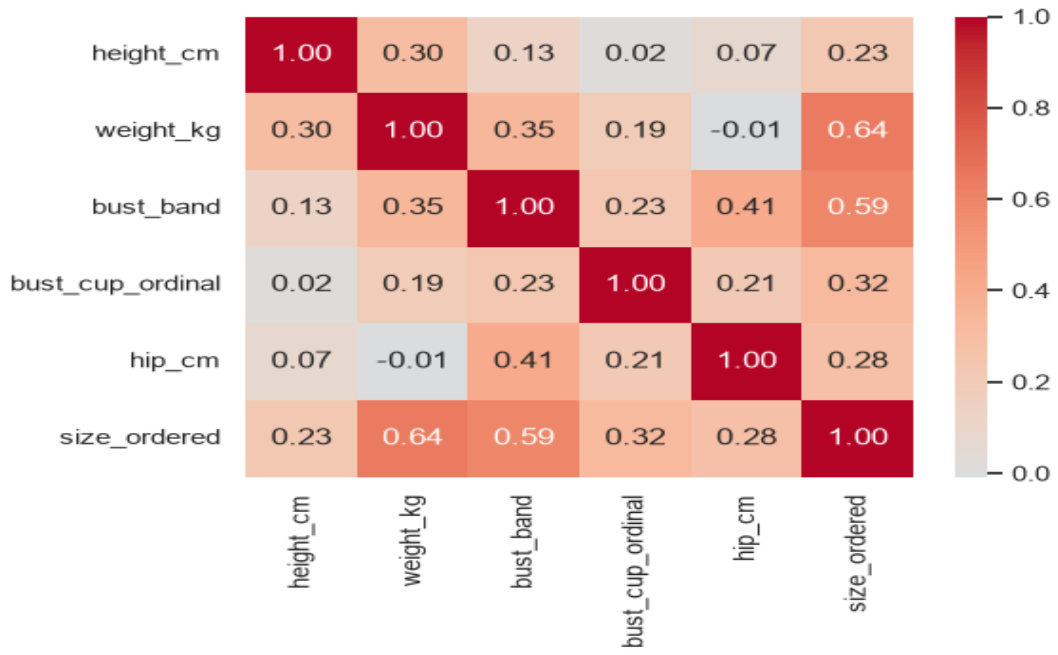

Figure 3. Correlation heatmap of numeric features.

size_ordered shows the clearest class separation (it is mechanically tied to fit), while body measurements separate weakly in isolation — fit depends on the relationship between measurements and the ordered size, not on measurements alone.

PCA and exploratory feature importance

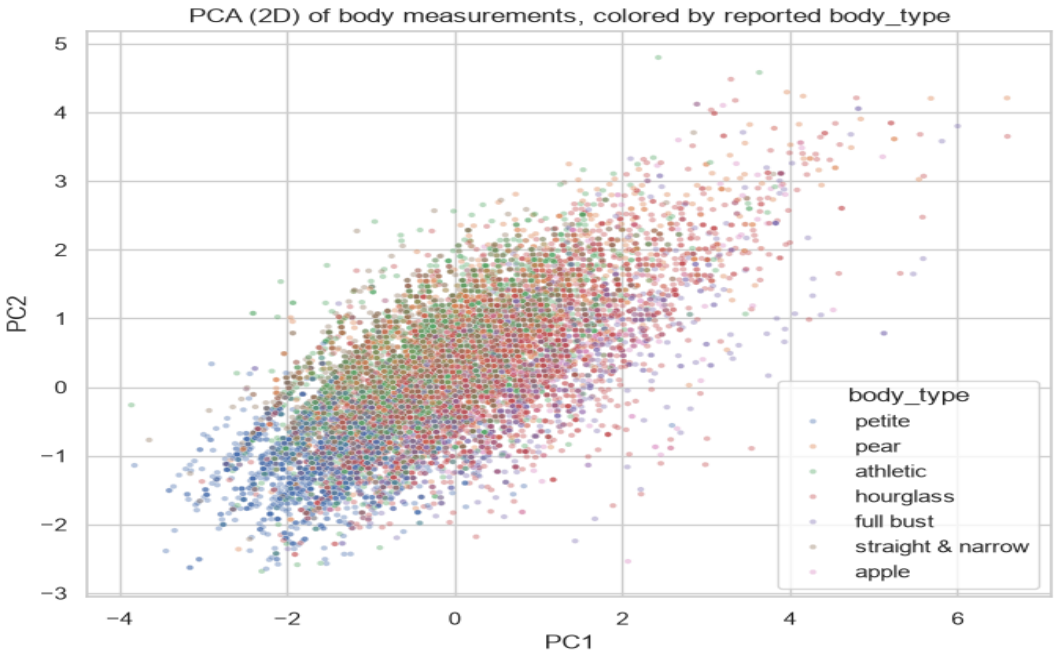

Figure 4. PCA (2D, visualization only) colored by body type.

Body-type clusters overlap substantially in 2D: PC1/PC2 are dominated by overall body size, while body-shape categories are defined by proportional relationships (e.g. waist-to-hip ratio). This is a useful negative result — raw measurements will not trivially separate body types, reinforcing why the fairness audit needs per-group metrics rather than assuming a simple boundary.

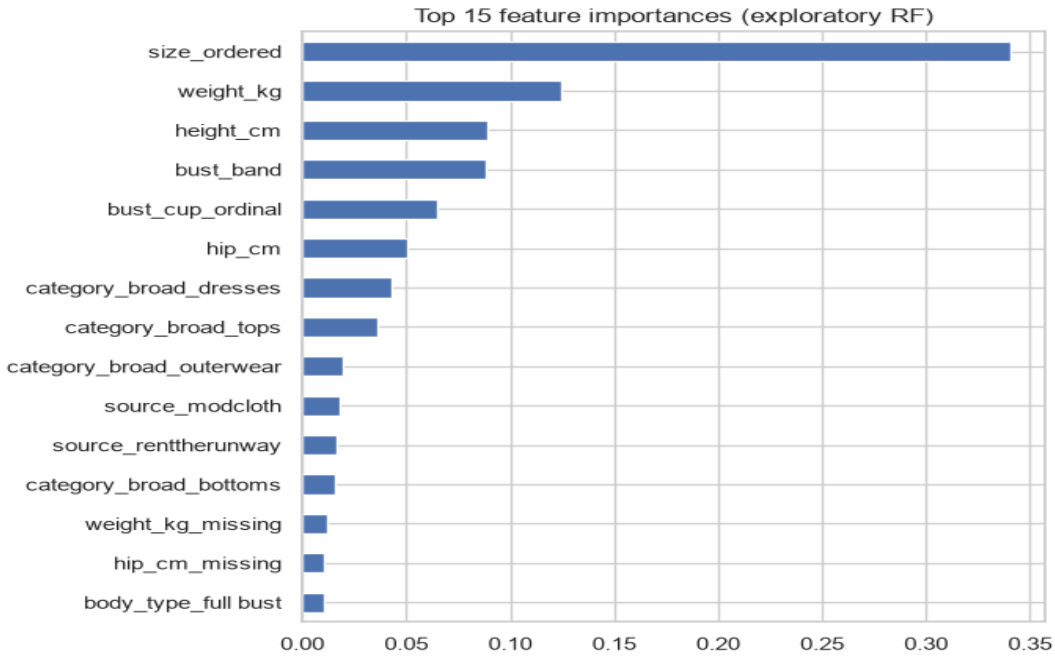

Figure 5. Exploratory random-forest feature importances (fixed seed).

Leakage discipline. model_ready.csv ships deliberately unscaled and not one-hot encoded: scaling and encoding are fitted inside each Phase 5 model's train-only sklearn Pipeline, so no test-split statistics ever leak into training.

4 Model Implementation & Comparison

Four classifiers — Logistic Regression (baseline), Random Forest, XGBoost, and a two-hidden-layer MLP (64/32, early stopping) — were trained by src/train_models.py on the same stratified 80/20 split (200,837 train / 50,210 test rows, seed 42), same features, same untuned equal-footing hyperparameters, with preprocessing fitted inside each pipeline on the training fold only.

model	accuracy	macro-F1	F1 fit	F1 large	F1 small	train time (s)
xgboost	0.7272	0.2918	0.8418	0.0219	0.0115	2.7
random_forest	0.7278	0.2848	0.8424	0.0101	0.0018	3.4
mlp	0.7272	0.2825	0.842	0.0043	0.0012	6.1
logistic_regression	0.7267	0.2822	0.8418	0.0	0.0046	0.8

Table 1. Phase 5 comparison (models/comparison.csv). Selected: XGBoost, by macro-F1.

Why macro-F1

Accuracy is statistically indistinguishable across all four models — each lands at the 72.7% majority baseline — which is precisely why it was ruled out as the selection metric. Macro-F1 weights each class equally, exposing minority-class failure: the “small” and “large” predictions are the entire point of a sizing assistant, because they are the cases where a recommendation prevents a return. XGBoost wins as the only model with non-trivial F1 on both minority classes.

The honest headline: unweighted models collapse to the majority class

Every unweighted model learns to say “fit” almost always (logistic regression never predicts “large” at all). Two causes: (1) with 73% of labels being “fit”, an unweighted loss is minimized by rarely risking a minority prediction; (2) the profile features carry real but limited signal — fit feedback also depends on garment-specific cut and sizing quirks no body measurement captures, and published work on this same dataset reports the task as hard for the same reason. This was deliberate for Phase 5: the unweighted models are the honest “before” for the fairness mitigation in Section 5.

Trees vs. the MLP — the expected finding, reported honestly

The Master Plan predicted tree-based models would beat the MLP on this modest-size, mixed-type tabular dataset, and they did: the MLP lands below both XGBoost and Random Forest on macro-F1 despite the longest training time. This matches the well-documented pattern that gradient-boosted trees dominate MLPs on tabular data of this scale — a legitimate comparison result, not a tuning failure. CNNs and diffusion models were explicitly scoped out.

Which model the product serves

The deployed /recommend-size endpoint serves the class-weighted XGBoost from the Section 5 mitigation, not the unweighted Phase 5 winner. For a sizing assistant the costly error is a missed misfit — the unweighted model gives wrong “it fits” reassurance in ~96–100% of misfit cases, while the weighted model catches roughly half of misfits (recall large 0.486, small 0.546). The weighted model is also better on the selection metric itself (macro-F1 0.3596 vs 0.2918). Its accuracy cost (0.727 $\rightarrow$ 0.396) is surfaced, not eliminated, by the product layer: the endpoint returns class probabilities, and the displayed confidence % plus a blue/amber advisory box communicate uncertainty, so borderline predictions surface as a sizing tip, never a bare verdict.

5 Critical Thinking, Ethical AI & Bias Audit

Full report in docs/fairness_report.md; every number below is generated by src/fairness_audit.py, which re-derives the exact Phase 5 split (seed 42) and asserts it matches the pinned test indices before computing anything. The audit covers the real women's model only — the men's synthetic extension is excluded because group metrics on a ~200-row generated population would measure properties of the random-number generator, not real-world disparity.

5.1 Setup: harms, groups, metrics

A “fit” prediction is the favorable outcome (a confident go-ahead at the usual size); small/large predictions route the shopper into size-adjustment friction. Two harms matter: quality-of-service harm (misfit detection failing more for some groups $\rightarrow$ more wrong reassurance $\rightarrow$ more returns) and allocation-style harm (the favorable prediction handed out at unequal rates). Groups audited, each with $\geq$ 1,000 test rows: body_type (8 groups; “not_reported” is the plurality at n=14,800 of 50,210 test rows, 29.5%), height_band (under 160 / 160–169 / 170+ cm), and size_band (US 0–6 / 8–14 / 16–22 / 24+). Metrics: per-group accuracy and per-class recall, disparate impact ratio

(four-fifths rule, <0.8 flagged), and equalized-odds TPR/FPR spreads.

Audit scope. Groups audited here are morphological proxies, not demographic protected attributes: the dataset contains no age, race, disability, or gender fields, and the population is women-only (both ModCloth and RentTheRunway serve women's fashion). body_type, height_band, and size_band are the fit-relevant dimensions this clothing-recommendation domain actually supports, and the audit stratifies on all three. Results measure disparity in model performance across these groups; they are not a substitute for an audit of demographic protected attributes, which the data does not contain.

5.2 Baseline audit: “fair” numbers that mean nothing

No group is DI-flagged at baseline (all ratios ≥ 0.98) — and that apparent fairness is vacuous. The unweighted model predicts “fit” for 99.5% of all rows in every group; a model that always says yes passes the four-fifths rule by construction. The metrics that look inside the predictions show real disparities:

– Accuracy tracks body size: size 0–6 shoppers get 0.7506, size 24+ get 0.6781 — a 7.3-point gap against plus-size shoppers, who experience misfit more often and get the least reliable model exactly there.

– Misfit detection is near zero for everyone, but unevenly so: full-bust shoppers get their “runs large” cases caught 8.2% of the time, pear-shaped shoppers 0.34% — a 24× ratio. The “small” class has literally zero recall for apple body types and size 0–6.

– The fit-class false-positive rate is 0.96–1.00 in every group: when an item does not fit, the model tells virtually every shopper that it does.

Conclusion: the baseline's bias problem is not unequal favorable-outcome rates; it is a degenerate predictor whose failures concentrate on the shoppers most likely to experience misfit. The mitigation must attack minority-class recall.

5.3 Explainability: SHAP on the winning model

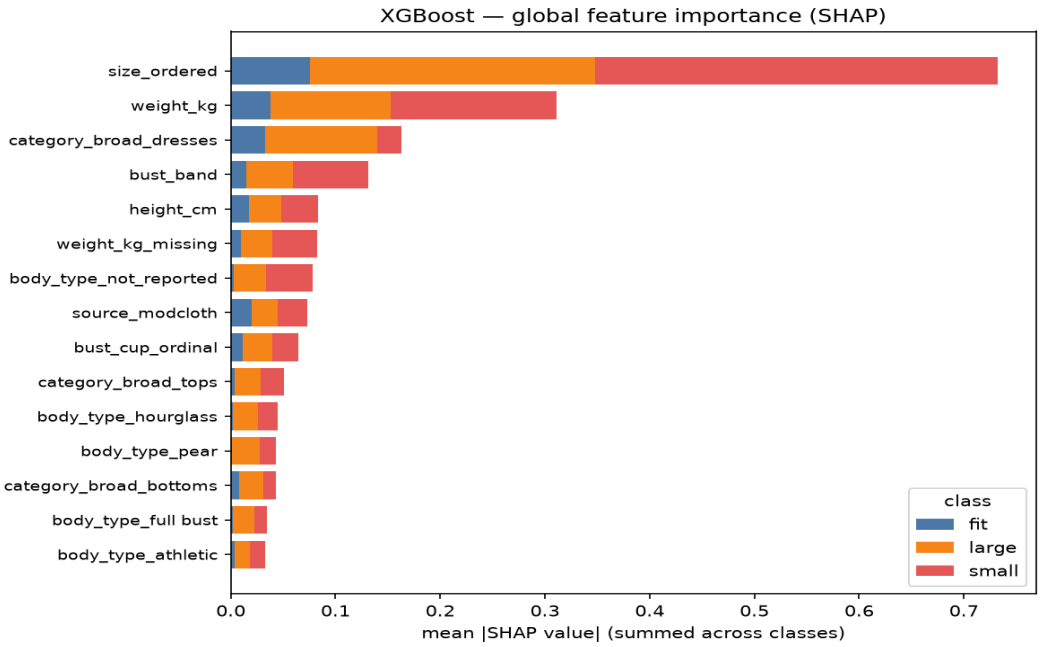

Figure 6. Global SHAP feature importance (TreeExplainer, 3,000 test rows, seed 42).

size_ordered dominates, especially for the minority classes (mean |SHAP| 0.38 for small, 0.27 for large vs 0.08 for fit), and the beeswarms read sensibly: ordering a high size, low weight, and a small bust band all push toward predicting “runs small”. The model has learned real, directionally correct sizing signal — it is the decision threshold, drowned by class imbalance, that fails.

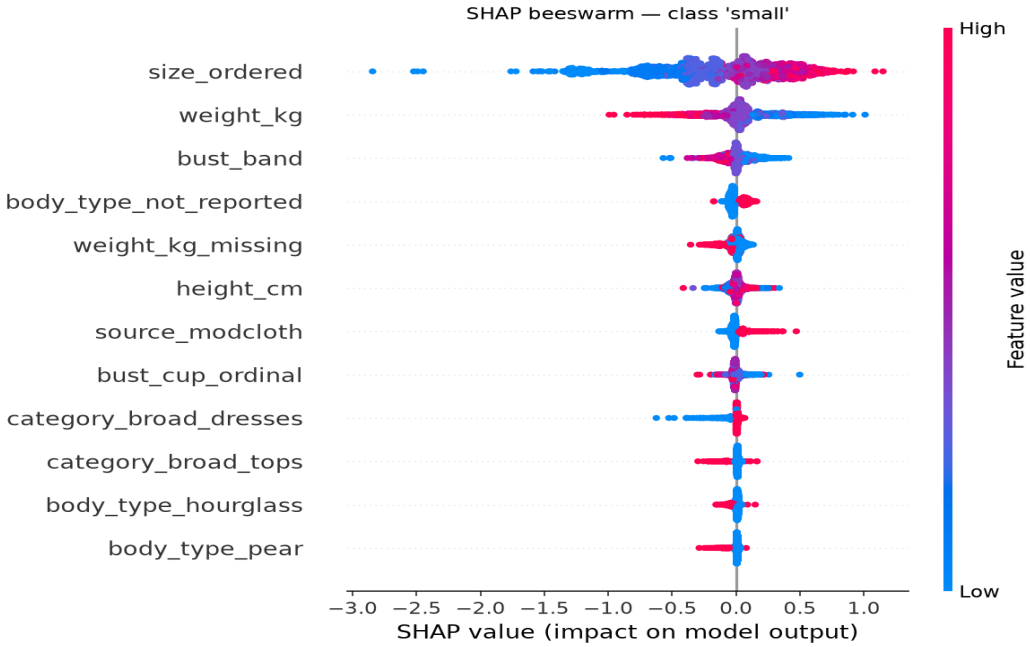

Figure 7. SHAP beeswarm for the “small” class.

Fairness-relevant red flag: missingness is predictive. weight_kg_missing and body_type_not_reported rank in the top seven features for minority classes. These overlap heavily with source_modcloth (weight is missing for all ModCloth rows): the model partly keys on which platform the shopper came from and how much they disclosed, not on their body. Shoppers who decline to report body type or weight get systematically different predictions — a data-completeness bias the product answers with transparency (the confidence box lowers displayed confidence when key measurements are absent).

5.4 Mitigation: class-weighted retraining, before/after

One mitigation, fully audited, as scoped: identical pipeline, identical train fold, identical seed — the only change is balanced sample weights (weight ∝ 1/class frequency) passed to XGBoost.

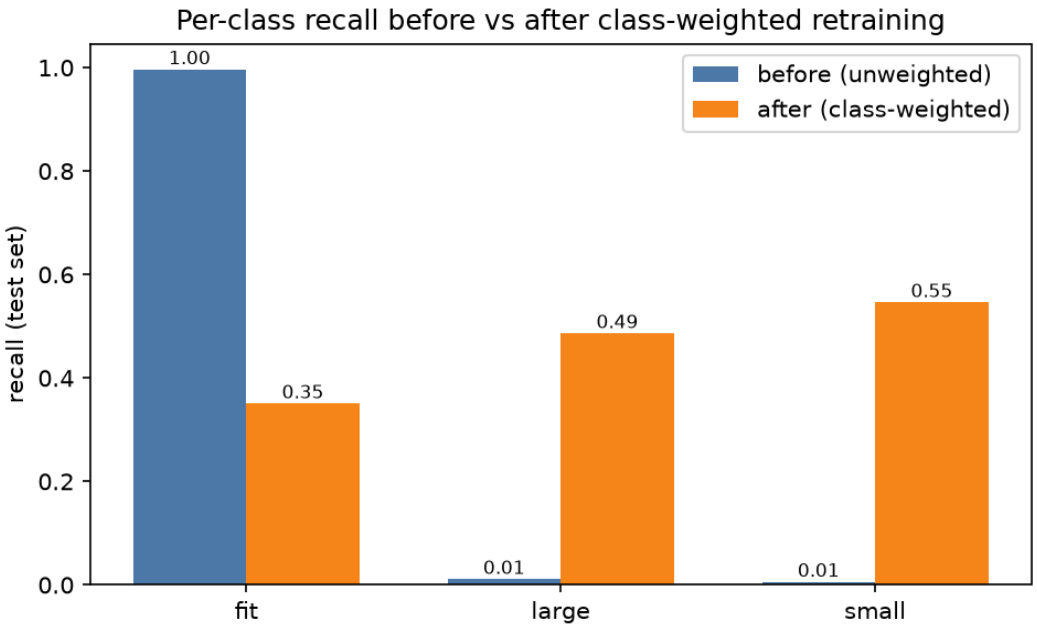

Figure 8. Per-class recall, before (unweighted) vs after (class-weighted).

metric	before (unweighted)	after (class-weighted)
macro-F1 (selection metric)	0.2918	0.3596
recall — large	0.0112	0.4864
recall — small	0.0059	0.5463
min per-group large recall (body_type)	0.0033	0.3553
min per-group small recall (body_type)	0.0000	0.4849
worst-group fit FPR (wrong "it fits")	0.9969	0.3463
accuracy	0.7272	0.3959
DI flags (body_type + size_band)	0 (vacuous)	7 (informative)

Table 2. Mitigation before/after — the fairness-critical numbers.

Reading the trade honestly: misfit detection now exists for every group (the wrong-reassurance rate dropped from ~0.96–1.00 to 0.13–0.35 everywhere), and macro-F1 rose 23%. The costs are equally explicit: accuracy fell to 0.396, most “it fits” cases now receive a spurious caution, and — importantly — disparate-impact flags appear. With the model actually discriminating between classes, the favorable prediction goes to size 8–14 shoppers at rate 0.42 but size 24+ at 0.17 (DI 0.41, flagged), and apple / full-bust / not_reported body types flag at DI 0.64–0.70. Height stays equal (DI ≥ 0.99). These flags are informative, not an artifact: the baseline's tiny spreads were a side effect of predicting one class everywhere, not evidence of equal treatment. The weighted model partly reflects real, higher misfit base rates in those groups, but at ratios well past the 0.8 threshold and with spurious “runs small” warnings (FPR up to 0.60) for size 16+.

5.4.1 Data-completeness as a fairness axis

Section 5.3 flagged weight_kg_missing and body_type_not_reported as top-seven SHAP features for the minority classes, overlapping with source_modcloth (weight is missing for all ModCloth rows). This raises an obvious question: do the DI flags above reflect real group disparities, or are they a data-collection artifact of which platform a shopper happened to use? A stratified check against the pinned test set answers it directly. Four of the five DI-flagged body_type groups — apple, full bust, hourglass, pear — are **100% RentTheRunway rows (0% ModCloth)**, because ModCloth never collected body_type at all; only the not_reported group is source-mixed (79.4% ModCloth, 20.6% RentTheRunway shoppers who declined to answer). The two DI-flagged size_band groups fare the same way: ModCloth's share of size 16–22 (20.7%) and size 24+ (25.5%) sits within a point or two of the unflagged bands (21.5% and 25.1%) and the overall test-set average (23.4%) — flat across the board, not concentrated in the flagged groups. weight_kg_missing is also not a pure ModCloth artifact: 15.9% of RentTheRunway rows have it too, and the rate still varies by body_type (8.1–21.7%) within the all-RentTheRunway groups. **Conclusion:** the disparate-impact flags on body_type and size_band are not primarily a platform-missingness confound — they reflect disparities the class-weighted model has genuinely learned within a single, same-source population. The one flag that is source-mixed (not_reported) is the one place this audit's missingness caveat still applies at face value. Mitigation (future work): stratified decision thresholds by data-completeness would still be the correct fix for the not_reported group specifically; the confidence box's lower displayed confidence when key measurements are absent is a disclosure today, not a calibrated mitigation.

Deployment stance. The product serves the class-weighted model: catching misfits matters more than raw accuracy for a sizing assistant. The confidence layer communicates uncertainty — a blue checkmark for confident matches, an amber “Sizing tip” box instead for borderline ones — so predictions surface as advisory, not bare verdicts. It does not eliminate the underlying false-positive rate: for size 16+ shoppers, up to 60% of “runs small” warnings are wrong (FPR 0.60, Table 2). The amber box contextualizes that false alarm; it does not remove it. A calibrated middle ground — probability calibration plus per-group decision thresholds tuned to equalize misfit-detection rates — is the clear next step, documented as future work to keep the before/after comparison clean.

5.5 Leakage checks and limitations

– Preprocessing fitted train-only inside pipelines; the audit asserts the re-derived split matches the pinned test indices; no post-purchase fields among features; size_ordered is legitimately known at prediction time.

– Residual risk, documented: the random split does not group by customer, so a repeat customer could appear on both sides. Customer IDs were dropped upstream; noted as a known limitation.

– Self-reported measurements are noisy, plausibly in group-correlated ways (social-desirability bias in weight reporting is well documented).

– The largest body_type group is “declined to say” (n=14,800), so per-group numbers for reported categories are conditioned on willingness to self-categorize.

– Bust input-method (band+cup native vs converted chest measurement) is a fairness-relevant data-quality axis the training data cannot audit; the product logs a per-user bust_input_method flag specifically so a future audit can stratify by it.

6 Deployment

The audited model (Section 5) is live end-to-end, not just trained and benchmarked. This section covers the deployment itself (6.1) and the product it serves (6.2).

6.1 Live end-to-end deployment

Live end-to-end on free tiers: the Flask backend runs as a Docker web service on Render (https://fit-ml.onrender.com) and the static frontend on Netlify (https://fit-ml.netlify.app). Free tiers spin down when idle — the first request after idle can take 30–60 seconds to cold-start.

– Docker was required because MediaPipe's Tasks API loads GPU-stack shared libraries (libGLESv2) even on the CPU delegate; the Dockerfile installs the needed system libraries that Render's native Python runtime cannot.

– Catalog images are gitignored, so a build-time script fetches them from a GitHub Release asset — keeping binary blobs out of git history while making assets available on the ephemeral-disk host.

– The full flow was verified against the live backend: photo upload (with pose extraction and the opt-in privacy crop) → size recommendation → generative try-on → Claude advice call.

– The ANTHROPIC_API_KEY lives only in the host's environment variables, never in the repository.

– Scope note: the live demo uses only the fairness-audited women's dataset and the class-weighted XGBoost model. The men's synthetic extension (Section 2, ~200 rows, deterministic size-chart lookup) is included in the repository as a scoping example but is not deployed on the live site; it is excluded from every fairness audit by design.

6.2 The FitML prototype: try-on, advice & privacy

A functional prototype website (not an online store — no checkout, payments, or inventory), live at https://fit-ml.netlify.app with six pages: Landing, About/How-it-works, Profile setup, Catalog, Try-on result, and History. Flask backend, plain HTML/CSS/JavaScript frontend (no framework), responsive from 375px mobile to wide desktop.

Generative virtual try-on

MediaPipe extracts pose keypoints from the user's uploaded photo; Claude Vision then analyzes the shopper photo and the target garment image and returns a structured JSON prompt (pose, garment placement, fit cues) that drives three garment-conditioned rendering engines, tried in order per request. **IDM-VTON** (garment-conditioned diffusion, via Replicate) is the default: it takes the shopper's photo and the actual catalog garment cutout directly, with masking and blending happening inside the model. On failure, the pipeline falls back to **SDXL inpainting** — a documented, root-caused limitation of this fallback: the community-maintained cog wrapper does not reliably implement mask-constrained inpainting the way the standard diffusers pipeline does, so it consistently renders a garment resembling the shopper's original clothing rather than the requested catalog item (tested against raising strength and guidance_scale to their maxima; both made results worse, not better, ruling out a simple parameter fix). On a second failure, the pipeline falls back to the original 2D affine-warp compositor (MediaPipe keypoints + alpha-composited garment PNG on HTML5 Canvas) as the final, always-available tier. Full 3D (cloth simulation or diffusion-based mesh rendering) was considered and ruled out as out of scope for a solo build at any tier. **Size-proportional rendering — a documented gap, not shipped as claimed in an earlier draft.** Only the final-fallback 2D compositor actually scales the garment render to a non-recommended size's chart ratios (an XL on a small-framed user renders visibly wider and longer than an M there). IDM-VTON and SDXL, the two tiers that serve the overwhelming majority of live requests, receive no size or scale signal at all — IDM-VTON's API takes only the two images, a text description, and a category, with no lever for numeric fit; SDXL gets a qualitative fit description in its prompt ("render slightly oversized, loose through the body"), which a diffusion model may or may not visibly honor, not a geometric scale factor. Closing this gap for real would mean either conditioning the generative tiers on a scale factor they do not currently accept, or dropping every non-recommended-size request to the 2D compositor — both out of scope for this submission and noted as future work.

Lower-body standardization (shipped). For any top/outerwear try-on on a full-body photo, the region below the waist is always standardized to plain black fitted leggings, regardless of the shopper's actual bottoms — a consistent, catalog-style presentation independent of what is out of frame for the garment being tried on. IDM-VTON implements this as two chained calls (top first, then a second call on the first pass's own output against a fixed leggings reference), at roughly 1.7× the single-pass latency. A symmetric upper-body version — standardizing the top above a bottoms try-on — was built and tested but is **disabled in production due to a root-caused issue**: two different reference garments and both chained-call orderings each surface an IDM-VTON boundary-bleed artifact between the two chained passes. Rather than ship visually incorrect output, this feature is documented as future work pending a fix to the chaining logic (see docs/genai_usage.md).

Size recommendation UX

Measurements are always manually entered (height, optional weight, bust via band+cup or converted chest measurement flagged lower-precision, hip, body type). The uploaded photo is for try-on pose only and never feeds the size model. The recommendation arrives in a confidence box: blue with a checkmark for confident matches; amber with a lightbulb “Sizing tip:” for borderline cases (near a size boundary + low-stretch fabric + fitted cut → recommend one size up, lower the displayed confidence). Advice text is two paragraphs: measurement-based reasoning, then a “Note:” paragraph of plain-language visual observations from the composite.

Design lesson: correlation is not causation

The first /recommend-size design scanned candidate sizes and picked the one maximizing P(fit) — and testing showed it inverts: a petite profile got XL at 90% “confidence”. Cause: in the training data, size_ordered tracks the customer's own body, so the model learned the between-customer correlation (larger sizes are ordered by larger people, who disproportionately report items running small), not the within-person effect of changing size. A (petite body, size 17) row is far out of distribution, and the model's counterfactual answer there is meaningless. The fix: anchor the size deterministically from measurements via the size chart, ask the model the in-distribution question (“does this size run small/fit/large for this body?” — literally what each training row records), shift one size if the top class is a misfit, then apply the borderline advisory layer. Verified sane: petite → XS, average → M, large-bodied → XL.

History-based suggestions (rule-based, not a second model)

Each try-on stores size shown, brand, fabric, and the user's fit feedback. When 2+ prior items from the same brand or fabric show consistent feedback, a one-line note is added to the advice prompt (“items from this brand have run true to size for you”) — a SQL lookup plus a conditional prompt line, deliberately not another trained model.

Privacy (full framing in docs/privacy.md)

– Uploads live under anonymous UUID session folders (backend/uploads/{session_id}/), never name/email-keyed, gitignored, session-scoped.

– 24-hour auto-deletion enforced in code (purge on startup, before every upload, and as a scheduled script); the deployed host's ephemeral disk makes redeploys a stricter version of the same guarantee.

– Crop-at-upload, opt-in (unchecked by default): a checkbox on the upload form removes everything above the nose — the upper face (forehead, eyes) — before the photo is ever written to disk, pose-extracted, or sent to any try-on engine; the lower face (mouth, chin, jaw) remains visible on this path. Left unchecked (the default), the photo is stored and processed with the face visible, and gets a defense-in-depth protection instead: a face bounding box is detected once and carried in pose.json so every generated try-on render gets the real face

region re-pasted over whatever the engine drew there, with a feathered edge — closing a benchmarked IDM-VTON tendency to occasionally regenerate a synthetic, unrelated face on full-body renders.

– Third-party processing disclosure: generative try-on sends the shopper's photo (already cropped, if that checkbox was used) to Replicate to render IDM-VTON/SDXL; Replicate never receives body measurements, name, or any other profile field.

– HTTPS-only transmission; consent notice at the point of collection stating what is collected, why, and the retention period.

– Framed against the Philippines' Data Privacy Act of 2012 (RA 10173): transparency, legitimate purpose, proportionality, retention limitation, and security — with consent for the try-on feature never reused for research or auditing.

7 Generative AI Usage (Step 9, optional)

FitML uses generative / deep-learning AI strictly outside the graded ML pipeline, in three places (full write-up in docs/genai_usage.md):

Claude API — multimodal fit advice

After the try-on composite renders, the /advice endpoint sends Claude Vision the model's pre-computed size recommendation, its confidence level, and the composited try-on image — never raw measurements. Measurements are always manually entered by the shopper; Claude cannot estimate or modify them. Claude returns two paragraphs of plain-language advice that can reference visual details in the composite (how the garment sits, whether it looks loose or snug). Hard boundaries: Claude receives the model's recommendation as read-only context and never overrides it, and its visual observations are qualitative commentary only. The prompt bans tailoring jargon so the output stays readable for non-technical shoppers.

Generative virtual try-on — Claude Vision + IDM-VTON / SDXL

Claude Vision analyzes the shopper's uploaded photo and the target catalog garment to generate a structured JSON prompt (pose, framing, garment description) — never measurements. That prompt drives IDM-VTON (garment-conditioned diffusion via Replicate, the default engine), which renders the composite showing the shopper wearing the catalog item; SDXL inpainting is the fallback if IDM-VTON fails, and the original 2D affine-warp compositor is the final, always-available tier (Section 6.2 has the full three-tier detail). The analyzer prompt is hard-banned from mentioning body measurements, enforced by an automated smoke-test assertion, and no data from this pipeline feeds the graded size model — it renders the recommended size, never estimates it. Seeds default to 42 and Replicate model versions are pinned in code for reproducibility, the same discipline the graded pipeline uses.

rembg / U²-Net — catalog background removal

The catalog pipeline runs product photos through rembg's clothing-segmentation model (U²-Net, a pretrained CNN) to produce the transparent garment PNGs the compositor warps. It runs offline, once per image, sees no user data, and touches no training data.

Why GenAI is fenced off from the graded pipeline

– **Auditability:** the fairness audit makes quantitative claims about a model whose inputs and parameters are fixed and inspectable; a generative model in the prediction path would make those claims unverifiable.

– **No silent data invention:** letting a vision model estimate measurements from photos — whether for advice text or for try-on rendering — would inject unquantifiable, likely body-type-correlated error into exactly the variables the audit stratifies on.

– **Reproducibility:** every graded artifact reproduces from fixed seeds and committed code; generative output is confined to surfaces where variation is harmless (advice prose, one-time asset prep, and seeded try-on rendering).

The result: deterministic, audited ML decides the size; generative AI only renders and explains it, and helps build the demo catalog.

8 Presentations (Step 6)

Two decks accompany this report, both following the program branding (white background, AIM logo on every slide):

– **Technical deck** (Debie_Galleros_Pillar5_CapstoneProject_TechnicalDeck.pptx, audience: peers and faculty): problem framing and task type, dataset overview, EDA highlights, the four-model comparison with macro-F1 reasoning, fairness-audit methodology with before/after mitigation numbers, SHAP explainability, the data → model → Flask → website architecture, GenAI usage boundaries, and limitations + future work.

– **Business deck** (Debie_Galleros_Pillar5_CapstoneProject_BusinessDeck.pptx, audience: executives, no jargon): the returns problem ($100B+, 35% online return rate), what FitML does in shopper language, the fairness differentiator, live-site screenshots, business impact and ROI logic, privacy and risk handling, rollout strategy, and future work.

9 GitHub Repository (Step 7)

https://github.com/debiegalleros/fitml-capstone — public, with the rubric-required structure (src/, notebooks/, data/, models/, backend/, frontend/, docs/, README.md, requirements.txt) and a clean, incremental commit history: one commit per meaningful unit of work with descriptive messages, phase by phase from scaffolding through deployment.

– **Reproducibility:** fixed seed 42 throughout; training config, label encoder, and pinned test indices committed alongside the model artifacts; the fairness audit refuses to run if the re-derived split stops matching the pinned indices.

– **Hygiene:** raw datasets, catalog image assets, uploaded photos, and API keys are gitignored; requirements.txt is frozen to the exact working versions; the one oversized artifact (random_forest.joblib, 50 MB) is gitignored and byte-reproducible from the fixed-seed training script.

– **Documentation:** eight docs/ files cover the data dictionary, model selection, the full fairness report, data provenance, privacy, GenAI usage, and deployment — all cross-linked from the README.

10 Limitations & Future Work

– Class imbalance remains the dominant modeling problem. Next steps beyond the single audited mitigation: probability calibration with per-group decision thresholds, focal loss, or resampling.

– Customer-grouped splitting: re-derive customer IDs upstream so repeat customers cannot straddle the train/test split.

– Fit-aware generative try-on: this submission already ships generative try-on (IDM-VTON primary, SDXL fallback), but size-proportional rendering (Section 6.2) only reaches the final 2D-compositor fallback tier — IDM-VTON and SDXL, which serve almost every live request, are not conditioned on a numeric scale factor today. The FIT dataset (2026; 1.13M fit-aware try-on triplets) is the cited research path beyond this: a model trained specifically to render how a given size actually drapes and fits, rather than garment-conditioned compositing at a scaled size — which would also close the scale-factor gap above.

– Per-item size charts are illustrative, not brand-specific: the "Size Measurements" table shown on every women's item detail page is one shared demo chart (Section 6.2's chart-anchor logic), not a real per-brand chart pulled from the catalog data — the catalog has no per-item chart data to pull from.

– SDXL's fork-specific weak prompt adherence (Section 7) is a documented, root-caused limitation of the fallback engine, not the primary one; a same-tier alternative inpainting backend is future work if IDM-VTON's Replicate availability ever becomes a concern.

– Symmetric upper-body standardization (mirroring the shipped lower-body leggings feature for bottoms try-on) was built and tested but stays disabled: two reference garments and both chained-call orderings each surfaced a genuine IDM-VTON boundary-bleed problem between the two chained passes.

– Transformer-based clothing parsing: a SegFormer-B2-clothes benchmark against the current U²-Net pipeline found no net quality advantage today (59 vs 56 clean items, 5 outright failures), but the approach is the natural upgrade as models mature.

– Color-suitability matching (does this shade complement the shopper) — identified from the start as a natural extension, deliberately out of scope.

– A CNN fit-quality assessor over the composited try-on image was considered and scoped out: no labeled dataset exists, and building one from the compositing pipeline was too time/risk-heavy for the deadline.

– Production hardening: pin CORS to the exact frontend origin, add per-group threshold monitoring, and collect the bust_input_method flag long enough to audit input-method bias with real usage data.

References & data sources

– Kaggle: Clothing Fit Dataset for Size Recommendation (ModCloth + RentTheRunway) — model training and fairness audit.

– Kaggle: Fashion Product Images Dataset (paramaggarwal, MIT license) — catalog imagery.

– Uniqlo published men's size charts (transcribed 2026-07-06) — men's extension label source.

– Republic Act No. 10173 (Data Privacy Act of 2012, Philippines) — privacy framing.

– FIT dataset (2026): 1.13M fit-aware try-on triplets — future-work research direction.

– Project repository: https://github.com/debiegalleros/fitml-capstone · Live demo: https://fit-ml.netlify.app